

INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

Contents

1	Introduction	2
1.1	Classification: predicting a category	2
1.2	Different classifiers fit different patterns	3
1.3	Digit images as points in a feature space	3
1.4	Training error, test error, and overfitting	4
1.5	Validation: choosing a model without using the final test	5
1.6	Supervised and unsupervised learning	6

LECTURE 1: INTRODUCTION

MACHINE LEARNING uses patterns in data to make predictions. A model describes the patterns we are willing to consider; statistics helps us reason about them; an optimization algorithm fits the model to the data. The amount and quality of the data limit what can be learned. More data can also make methods effective that worked poorly with smaller collections.

The central question is whether a learned pattern works on new examples. Remembering the examples already seen is not enough.

1.1 Classification: predicting a category

Definition 1.1 [CLASSIFICATION]. CLASSIFICATION predicts which category, or CLASS, an input belongs to. A LABEL records the known class of an example. A CLASSIFIER is a rule that assigns a predicted class to an input.

Example: *Credit card default.* For each cardholder, record the balance and annual income, together with whether the cardholder defaulted. These labeled records are TRAINING DATA. Use them to learn a rule that predicts default for a new applicant. The two classes are “default” and “no default.” In this example, balances distinguish the groups more clearly than incomes do, although the groups still overlap.

The measured inputs are FEATURES. Collecting them in a FEATURE VECTOR makes each example a point: two features give a point in a plane. A DECISION BOUNDARY separates regions where the classifier predicts different classes. It separates *predictions*; it need not separate every observed label correctly.

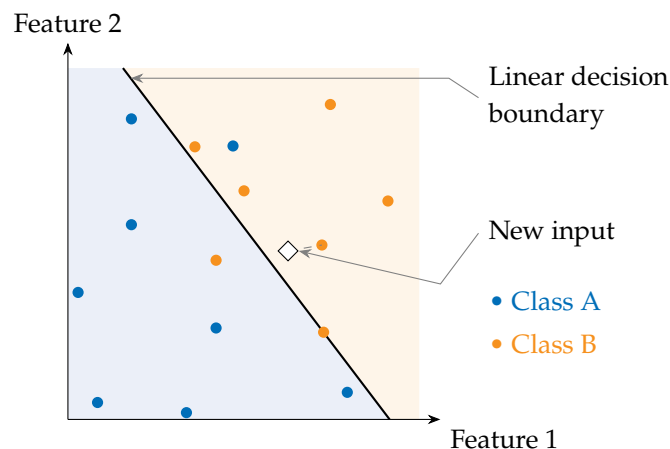


Figure 1: A schematic classification problem. The colored points have known labels; the diamond is a new input. One rule uses the nearest labeled point, while another uses the side of a straight boundary.

1.2 Different classifiers fit different patterns

The same training data can support different prediction rules.

Definition: The 1-NEAREST NEIGHBOR CLASSIFIER finds the training point closest to a new input and returns its label. The k -NEAREST NEIGHBOR CLASSIFIER instead finds the k closest training points and lets their labels vote. For two classes and odd k , one class has a majority.

At $k = 1$, the rule can respond to each individual training point. At $k = 15$, a point's label is balanced against fourteen other labels. This can give less intricate prediction regions, but may misclassify some training points. With distinct training inputs, 1-nearest neighbor classifies each training point correctly when that point itself is included among the possible neighbors: its distance to itself is zero. Conflicting labels at identical inputs would prevent this conclusion.

A LINEAR CLASSIFIER uses a straight decision boundary in two dimensions. This is a stronger restriction on the shape of the pattern. It can miss genuine nonlinear structure, but it also avoids the small islands that a nearest neighbor rule can create around individual points.

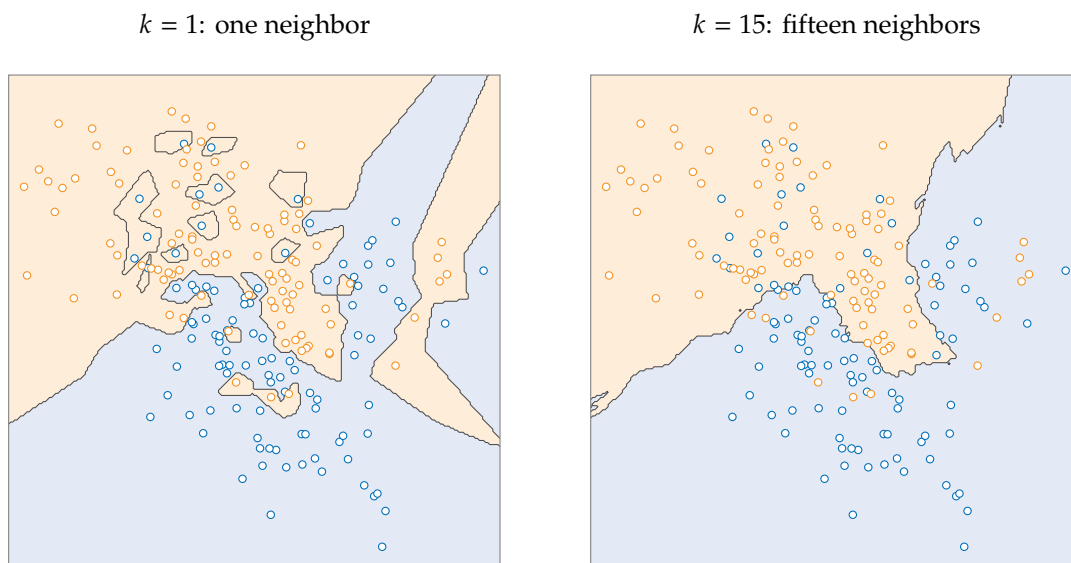


Figure 2: The same 200 labeled points classified with $k = 1$ and $k = 15$. Background colors show predictions; circles show the training labels. At $k = 1$, small regions follow individual points. At $k = 15$, votes produce broader regions, and some circles lie in a region of the other color.

Correctly classifying all training points is therefore not enough to choose a rule. A very intricate boundary may capture accidental details rather than a pattern that will persist in new data.

1.3 Digit images as points in a feature space

Example: *Handwritten 7s and 1s.* Read the sixteen pixels of each 4×4 image in a fixed order to obtain a point in $\mathbb{R}^{16}$. Labeled images of the two digits train a classifier.

4 × 4 pixel array

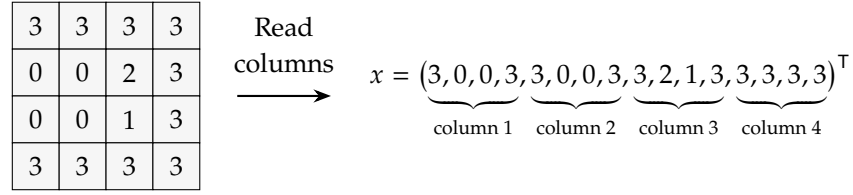


Figure 3: A 4 × 4 array becomes a sixteen-coordinate vector by reading down each column, from left to right. Training and new images must use the same coordinate order.

A **HYPERPLANE** is the higher-dimensional version of a straight line boundary. Thus a linear classifier for these images has a hyperplane as its decision boundary in sixteen-dimensional feature space.

The classification pipeline has two stages:

1. *Training*: compute feature vectors for the labeled images, then learn a classifier from the vectors and their labels.
2. *Prediction*: compute the feature vector of a new image in the same way, then evaluate the learned classifier on that vector.

1.4 Training error, test error, and overfitting

Definition 1.2 [TRAINING AND TEST ERROR]. The **TRAINING ERROR** is the fraction of training examples that the classifier labels incorrectly. The **TEST ERROR** is the fraction labeled incorrectly in a separate collection of new examples that were not used to train the classifier.

If a labeled set contains n examples, the error rate is

$$\text{error rate} = \frac{\text{number of incorrect predictions}}{n}.$$

Training error measures performance on familiar inputs. Test error checks **GENERALIZATION**: performance on new examples. The zero training error of 1-nearest neighbor says little by itself about generalization.

Definition 1.3 [OUTLIERS AND OVERFITTING]. An **OUTLIER** is a point whose label is atypical for its location, such as a borrower who appears solvent but defaults anyway. **OVERFITTING** occurs when sensitivity to outliers or other accidental patterns makes performance on new data worse.

A decision boundary should capture useful patterns. Conversely, **UNDERFITTING** occurs when the rule is too restrictive to capture the useful pattern. Very large k can ignore local distinctions; very small k can react too strongly to individual examples.

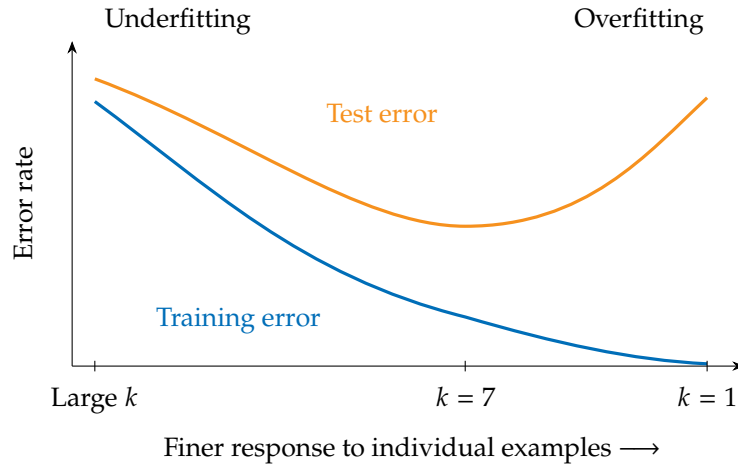


Figure 4: Schematic error trends as k decreases and the rule responds to finer details. Training error can keep falling while error on new data rises. The illustrated comparison has its best test performance near $k = 7$; this is an example, not a universal choice of k . No measured error rates are encoded by these curves.

1.5 Validation: choosing a model without using the final test

Definition: A **HYPERPARAMETER** is a setting that controls how a model is trained or how flexible its prediction rule is. The neighbor count k is one example. **VALIDATION** uses held-back labeled data to choose hyperparameters or compare models.

For a choice such as k , proceed as follows:

1. Hold back a validation set from the labeled data.
2. Train candidate classifiers with different settings, using the training set.
3. Compare their validation errors and choose the setting that performs best on the validation set.

The three sets have different jobs:

Set	Purpose
Training	Learn the model's fitted values, such as its weights.
Validation	Choose settings such as k , or choose among models.
Test	Evaluate the final choice once, after the choices are settled.

Keep the test set out of tuning. If its results influence model selection, it no longer provides the intended final check. A plot of test error against k can explain overfitting; choosing k in a new experiment requires validation data.

Prediction competitions illustrate the same issue. Repeated choices can be guided by public feedback, while a private set supplies the final score. Much stronger public results than private results can indicate overfitting to the feedback used during development.

1.6 Supervised and unsupervised learning

SUPERVISED LEARNING uses examples with target outputs. In classification, the target is a category. In REGRESSION, it is a numerical value. UNSUPERVISED LEARNING seeks structure in the inputs without supplied target labels.

Task	Example question
Classification	Is this email spam?
Regression	How likely is this patient to have cancer?
CLUSTERING	Which DNA sequences are similar to each other?
DIMENSIONALITY REDUCTION	What are common features and differences among faces?

Clustering groups similar inputs. Dimensionality reduction represents important variation with fewer coordinates. Both concern structure in the inputs rather than predicting a supplied label.

Revision. Classification predicts a category from a feature vector. A decision boundary describes where the prediction changes. Nearest neighbor rules and linear rules impose different shapes on that boundary. Generalization is the goal: training accuracy alone cannot establish it.

Common mistakes: choosing a classifier only because its training error is zero; assuming smaller k always helps; changing the pixel order between training and prediction; or using the final test set to tune a model.

Check yourself: Why can 1-nearest neighbor have zero training error and still overfit? Why does a 4×4 image have sixteen coordinates? Which set chooses k , and which set evaluates the final choice? What distinguishes supervised from unsupervised learning?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 1: Introduction](#), PDF pp. 2–6. Explanatory steps are completed and figures redrawn. The neighbor comparison uses the 200 plotted points on PDF p. 4 (originally ESL, Figures 2.1–2.3); error curves are schematic. Administrative material and unrelated text within textbook crops are omitted.